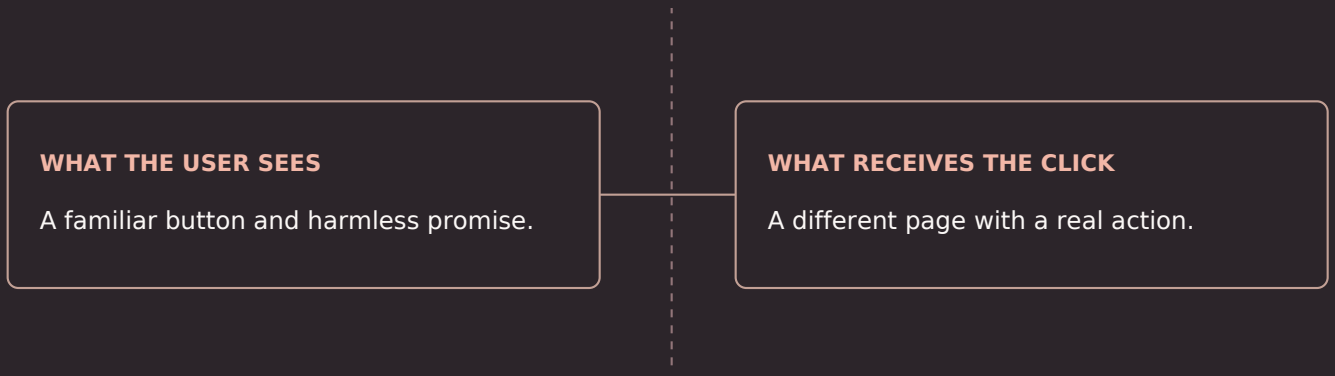
PERCEPTION / INPUT / INTENT

Clickjacking Explained

Follow the visible cue, trace the real click, and defend the action.



See why genuine input can carry false intent.
Prove the effect with a real browser and fictional data.
Verify the protection without breaking approved embeds.

Haroon Shahid

First edition / Version 1.0 / 4 November 2026

Before you begin

Clickjacking Explained

Follow the visible cue, trace the real click, and defend the action

By Haroon Shahid. First edition, version 1.0. 4 November 2026.

Copyright 2026 Haroon Shahid. The purchased publication is licensed for personal use under 05-ABOUT/Publication_License.txt. The separately identified lab code has its own license. Prepared with AI assistance. For corrections: gaknt2@gmail.com. The free preview may be shared unchanged with attribution. No resale or public redistribution of the paid publication is granted.

This book teaches responsible analysis through fictional examples and bounded local practice. It does not certify expertise. Use only systems and accounts you own or are explicitly authorized to assess.

The validation report identifies executed tests and limits. Framework and protocol guidance is version-sensitive. The local lab uses real browser frames, cookies, input, policy headers, and a fictional account. It is not a production security library.

PDF code can soft-wrap for page fit. The offline HTML and source preserve original lines. Responsive HTML and scalable diagrams provide an alternative reading format; no full PDF accessibility conformance is claimed.

Free preview: introduction, roadmap and Chapters 1-3. The full package adds the complete curriculum, answered reviews, workbook, reference sheets and Framewright Lab.

Contents

Before you begin 2
Contents 3
Introduction 4
Learning roadmap 5
01. One click, two meanings 7
02. The web exchange behind a button 9
03. From HTML to what appears on screen 11
Appendix A. Answer explanations 13
Sources and further reading 14

Try this while you read

After Chapter 3, draw one button twice: once as the user sees it and once as the browser hit-tests it. Mark who owns each layer. The full workbook turns that sketch into a real browser experiment.

Introduction

You press a button that says “Preview sample.” An account setting changes instead. The browser did not invent a click. The server did not accept a forged identity. You supplied a genuine gesture, and the application received a normal authenticated form submission. The missing piece was a reliable link between what you saw and what the gesture meant.

This book teaches that missing piece. We will follow a single fictional account from page layout to browser hit testing to a verified state change. Then we will compare a blocked frame, an approved partner, a nested unapproved ancestor and a protected approval. The same method works when a workflow has two steps, a prefilling parameter, a session constraint, or a context that changes over time.

The four observations

Cue. Hit target. Authority. Effect. Describe the visible or announced cue. Trace where the browser delivered the input. Identify what made the real operation eligible. Check the result independently. If any observation is missing, state what you still need to prove.

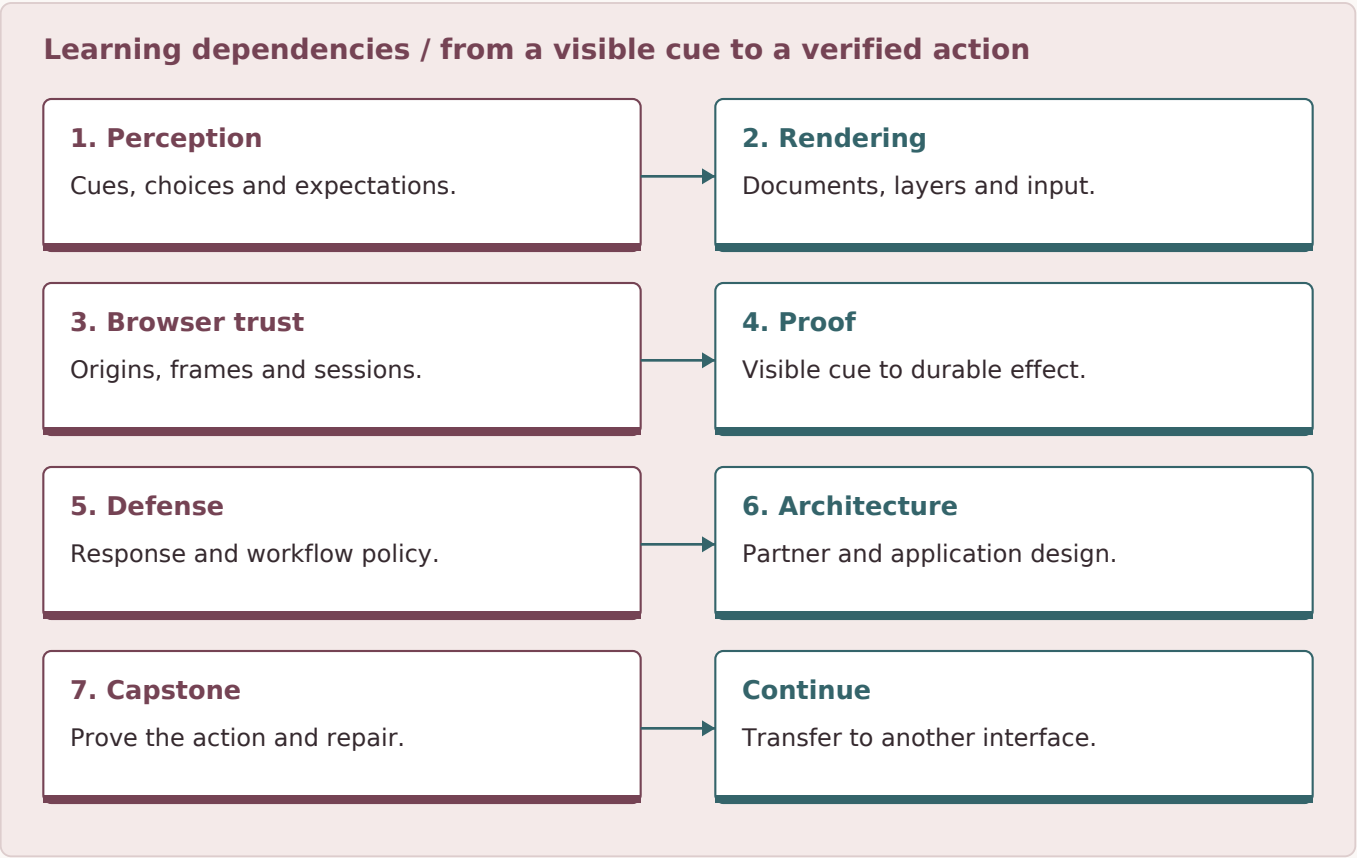
Framewright Lab uses real browser input over loopback. It keeps the effect harmless: a fictional theme changes. The workbook also covers analysis and design questions that the local fixture does not try to reproduce, including top-level window variants, immersive UI and deployed framework differences.

Learning roadmap

Route	Start here	Show you can...
First-time learner	Chapters 1–12	Explain how painted pixels, hit testing, origin and session can disagree.
Web security tester	Chapters 13–24	Reproduce a real input path and prove its effect with controls.
Builder or reviewer	Chapters 25–36	Protect an approval and keep a permitted partner integration working.

Gate	Observable milestone
Perception	Draw the cue and the real action separately.
Browser	Calculate site, origin and the full ancestor chain.
Practitioner	Compare an ordinary click, hidden-frame click and pointer-pass control.
Advanced	Prove both a nested denial and a legitimate partner success.
Transfer	Design a protected approval for another surface and identify new tests.

Each chapter includes a primary diagram, a compact revision diagram, three questions with explained answers, a practice prompt and a worked example. Use the workbook to predict the outcome before you open the result.



Read each row left to right, then move down. Each later stage uses the earlier mental model.

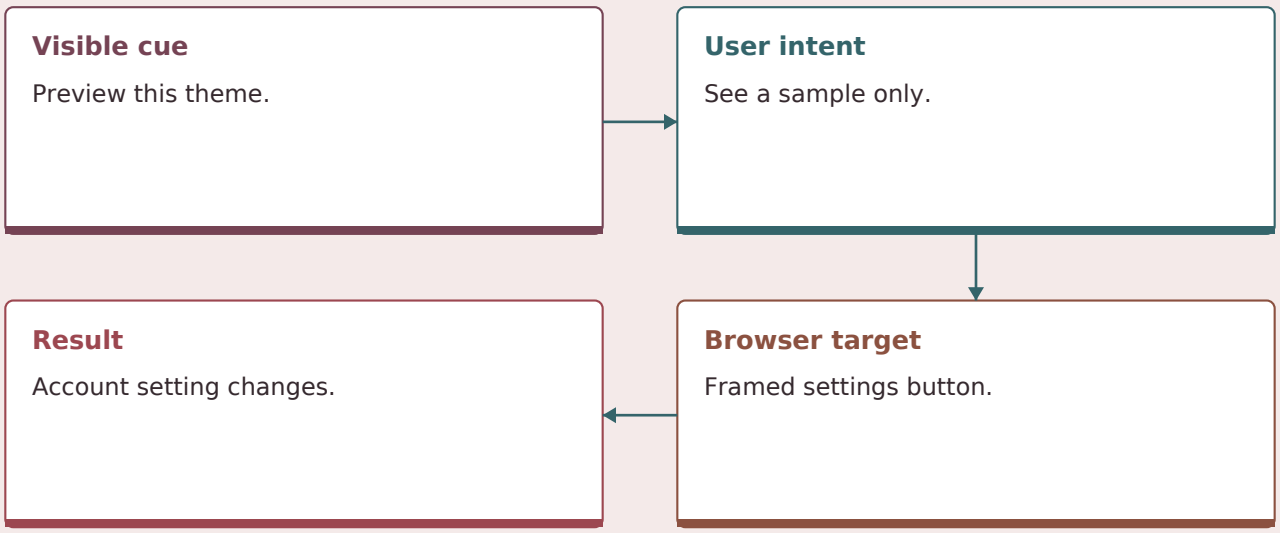
I. UNDERSTAND WHAT THE USER SEES

01. One click, two meanings

Learn to: Separate the visible cue, intended choice, input target and committed effect.

Think of it this way: A sign says Ring for help, but a hidden switch behind it opens a gate.

01A / A single gesture can have two stories



Name the visible invitation and the real destination independently.

The showroom example

Imagine you are signed into Harbor Settings. Another page offers a theme preview. You see a button labeled “Preview sample,” but a transparent frame above that button contains your real account settings. When you click, the browser delivers the input to the settings control. It submits a normal request and saves your theme. Your click was genuine; the meaning you assigned to it came from the surrounding page.

UI redressing means presenting an interface in a way that misleads the person about the action they are about to take. Clickjacking is a familiar instance. The frame is a mechanism. The deeper problem is losing the relationship between visible context, input and the action that uses the session.

Start with a concrete mismatch

Write one sentence the user could honestly say: “I thought I was seeing a preview.” Write one statement the server can verify: “The signed-in fixture submitted a valid settings form.” These statements can both be true. A security design needs to protect the connection between them, especially when the action has consequences.

Practice

Draw three columns: human, browser, application. Put the apparent preview, actual button, and saved state in their proper columns.

01B / Cue, hit target, authority, effect

See	Believe	Target	Prove
Preview label	No account change	Real form control	Journal records save

Key takeaway. A real gesture is not sufficient evidence that the user understood the action it triggered.

Check your understanding

1.1 What four things should you record?	**1.2** Is every unexpected click a vulnerability?	**1.3** Is clickjacking mainly about reading another page?

Answer explanations: Appendix A. Sources: C01, C02, C03

WORKED EXAMPLE / Explain the mismatch in one sentence

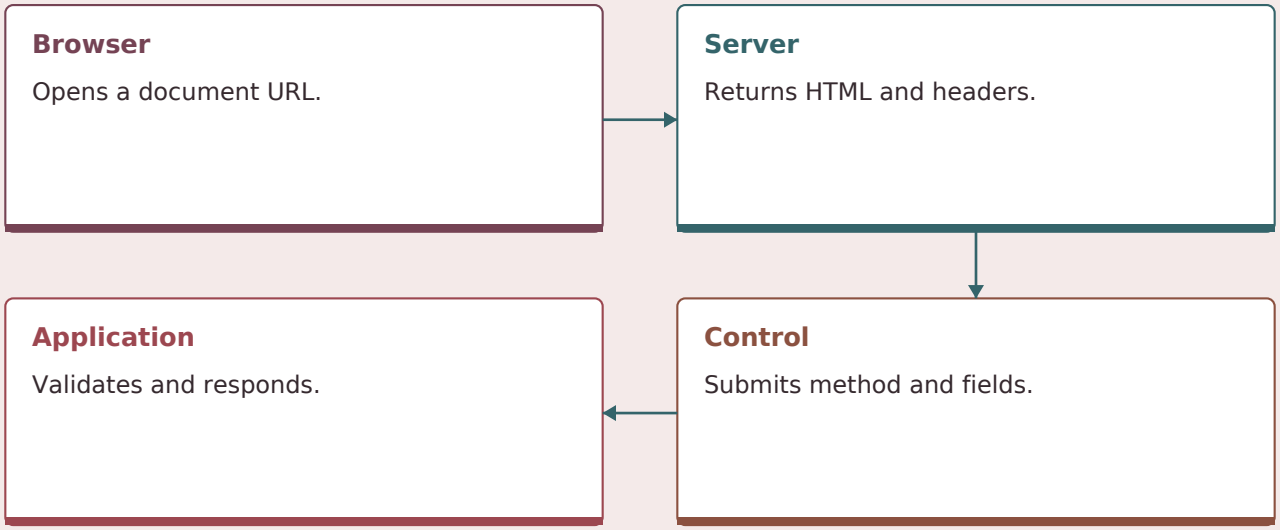
“The visible control invited a preview, but the browser activated an account form that recorded a change.” This sentence separates the human claim from the application evidence. If you can only say that an iframe returned 200, you have not yet explained a click or effect.

02. The web exchange behind a button

Learn to: Explain URLs, HTTP requests and responses, HTTPS and ordinary form submission.

Think of it this way: A store order form travels to the store named on it, even if someone holds the clipboard for you.

02A / The click becomes a web exchange



The action follows the target document's form rules.

How a page reaches the screen

The browser connects to a server to get a document. A URL identifies where the request goes and which resource it asks for. HTTP carries a method, request headers, and sometimes a body. A response carries a status, headers, and a body such as HTML. HTTPS adds protected transport. That protects the bytes in transit; it does not decide whether another page can embed the document.

A form can submit data without JavaScript. A submit button activates the form, and the browser sends the target URL, method and selected fields. A setting change should generally use a method intended for state changes, with server-side authentication, authorization and request validation. That is still not a test of how the user saw the form.

Why response headers matter

The browser can evaluate an embedding rule from the document response before presenting the sensitive UI. That rule travels with the target response even when a parent page wrote the iframe. Check the exact document that renders the control. A header on an API response or an unrelated home page does not automatically protect a different HTML response.

Practice

Identify the form method, target URL, browser policy and state-changing result in one local request. Do not confuse a request header with a response policy.

02B / Cue, hit target, authority, effect

See	Click	Request	Effect
A web document	Form control	Method and fields	Stored or denied

Key takeaway. HTTP identifies a request and its context, but a normal form submission does not describe the visible story around the button.

Check your understanding

2.1 What is a response header?	**2.2** Does HTTPS stop a separate HTTPS page from framing you?	**2.3** What happens when a button submits a form?

Answer explanations: Appendix A. Sources: C04, C05, C06

WORKED EXAMPLE / Protect the document that serves the button

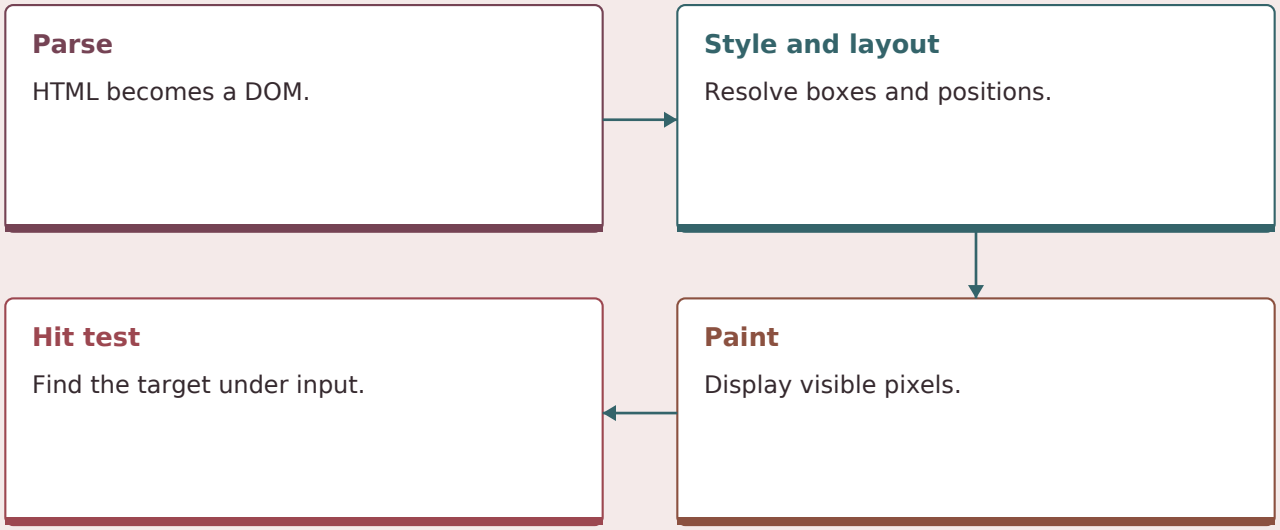
A storefront sends CSP on /api/config but omits it on /account/settings. The browser presents the HTML document, so the JSON response policy does not supply a missing ancestor rule. Inspect the document that actually renders the sensitive control.

03. From HTML to what appears on screen

Learn to: Follow parsing, style, layout, painting and input targeting as separate steps.

Think of it this way: A theater builds a set, lights the visible surfaces, then decides which actor is standing at a marked spot.

03A / Rendering is a process, not a screenshot



Style and layout can change after the first paint.

A beginner's rendering map

HTML describes elements: headings, controls, links and embedded documents. The browser parses it into a document object model, or DOM. CSS supplies style rules. Layout computes positions and sizes. Painting produces the visual parts, and the browser may combine layers for display. Exact scheduling differs by engine and can change as resources load.

A browser also has to decide where input goes. When the user clicks a point, hit testing considers layout and interaction rules to identify the receiving element or frame. A screenshot captures the painted scene. It does not tell you which control was eligible to receive pointer input.

Frames are another document

An iframe has a box in the parent document and a separate child document inside it. The parent can position that box without reading the child's internal DOM. This is why a tester needs both an alignment view and a real input test. If you directly call the inner button from automation, you skipped the question of whether the user's point would reach it.

Practice

Explain how one browser window could show a button from the parent document while hit testing routes the click into a child.

03B / Cue, hit target, authority, effect

Structure	Geometry	Pixels	Input
DOM and frames	Where boxes land	What becomes visible	Who receives it

Key takeaway. The pixels you see and the object selected for input are related but different browser results.

Check your understanding

3.1 What is the DOM?	**3.2** What does layout do?	**3.3** Is a compositor layer a security boundary?

Answer explanations: Appendix A. Sources: C05, C07, C08

WORKED EXAMPLE / Compare paint and input

At the preview point, the visible pixels are drawn by the parent. The top eligible box is an iframe. A parent hit-test query can identify the frame element without revealing the cross-origin child button. A coordinate click plus a journal entry completes the chain.

Appendix A. Answer explanations

Chapter 01: One click, two meanings

1.1 What four things should you record? Visible cue, intended choice, actual input target and independently verified effect.

1.2 Is every unexpected click a vulnerability? No. Establish who controlled the context, which boundary was crossed, and what meaningful effect occurred.

1.3 Is clickjacking mainly about reading another page? No. It often aims to make the other page act through its own real control.

Chapter 02: The web exchange behind a button

2.1 What is a response header? Metadata supplied with the response, before its body, such as a browser policy.

2.2 Does HTTPS stop a separate HTTPS page from framing you? No. HTTPS protects the transport and authenticates the endpoint; framing needs a separate policy.

2.3 What happens when a button submits a form? The browser builds a request from the form's method, action URL and successful controls.

Chapter 03: From HTML to what appears on screen

3.1 What is the DOM? The browser's document tree and objects, created from markup and later changes.

3.2 What does layout do? It computes sizes and positions for boxes under the current styles and available space.

3.3 Is a compositor layer a security boundary? No. Compositing helps render pixels; origin and browser policy govern security boundaries.

Sources and further reading

Research cutoff: 4 November 2026. The catalog distinguishes consulted material from targeted further reading. It does not claim every linked page was audited end to end or every client was deployed. Live documents change. Historical fixes are not current release recommendations. Original diagrams and synthetic exercises do not imply vendor endorsement.

C01 - Huang et al.: Clickjacking, Attacks and Defenses

Consulted. Context integrity and spatial or temporal deception; distinguish research designs from shipped controls.

https://www.usenix.org/conference/usenixsecurity12/technical-sessions/presentation/huang

C02 - OWASP Clickjacking Defense Cheat Sheet

Consulted. Browser response policies, cookie conditions and frame-busting limitations.

https://cheatsheetseries.owasp.org/cheatsheets/Clickjacking_Defense_Cheat_Sheet.html

C03 - PortSwigger: Clickjacking

Consulted. Mechanisms, controlled practice and multi-step examples.

https://portswigger.net/web-security/clickjacking

C04 - RFC 9110: HTTP Semantics

Further reading. Requests, responses and methods; HTTP success is not an embedding decision.

https://www.rfc-editor.org/rfc/rfc9110.html

C05 - HTML Living Standard

Consulted. Navigables, iframe attributes and sandbox; see the standard's forms and interaction chapters for related behavior.

https://html.spec.whatwg.org/multipage/iframe-embed-object.html

C06 - MDN: The HTML form element

Consulted. Native form action, method and submission.

https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/form

C07 - web.dev: How browsers work

Consulted. Rendering overview. Browser engines differ in scheduling and internal details.

https://web.dev/articles/howbrowserswork

C08 - CSSOM View: elementFromPoint

Consulted. CSS boxes and hit-test observations.

https://drafts.csswg.org/cssom-view/#dom-document-elementfrompoint